π Calculator



By Ianis Donica

Why a custom BigInt/BigFloat class

Why a custom BigInt/BigFloat class

The algorithm for computing Pi produces **insanely large** numbers

Why a custom BigInt/BigFloat class

The algorithm for computing Pi produces **insanely large** numbers

We may want to compute Pi for very very large digit counts

- PI is known up to **314 trillion digits**
- uint64 is only **4 Billion**

Why a custom BigInt/BigFloat class

The algorithm for computing Pi produces **insanely large** numbers

We may want to compute Pi for very very large digit counts

- PI is known up to **314 trillion digits**
- uint64 is only **4 Billion**

Conclusion: **We need types able to support arbitrary value ranges**

Big Int

Big Int

It is just a vector!

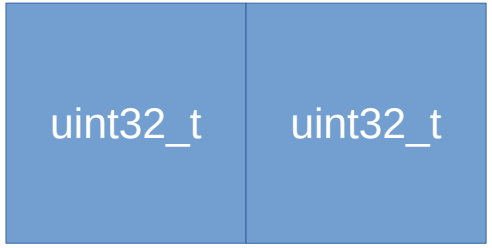
Big Int

It is just a vector!



Big Int

It is just a vector!



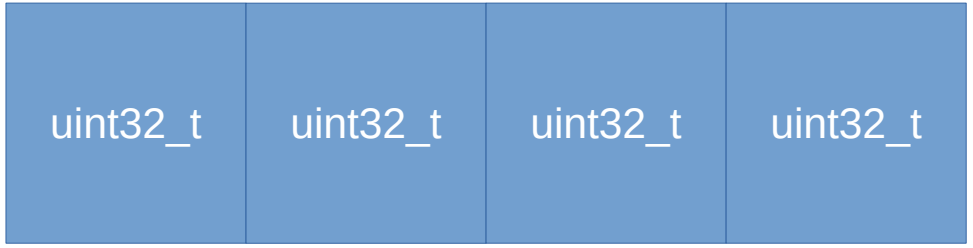
Big Int

It is just a vector!



Big Int

It is just a vector!



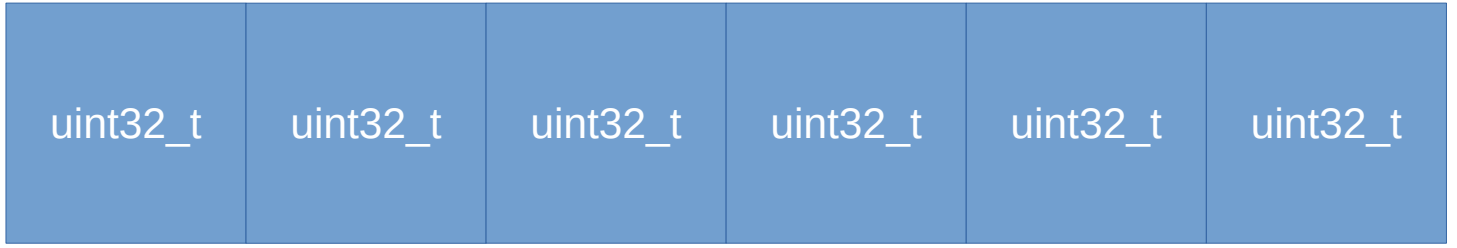
Big Int

It is just a vector!



Big Int

It is just a vector!



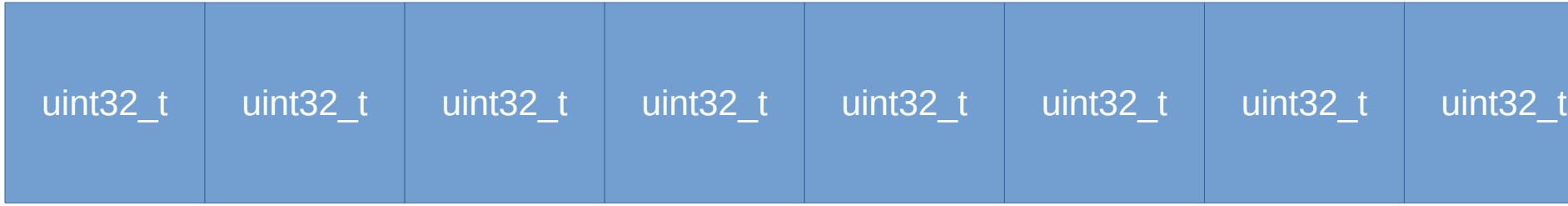
Big Int

It is just a vector!



Big Int

It is just a vector!



Big Int

It is just a vector!

(and a sign flag)



Big Int

It is just a vector!

(and a sign flag)





Each element is a chunk of 9 base 10 digits;
ranging from 0 – 999,999,999

Why only 32 bits?

Big Int Operations

Big Int Operations

Addition is simple:

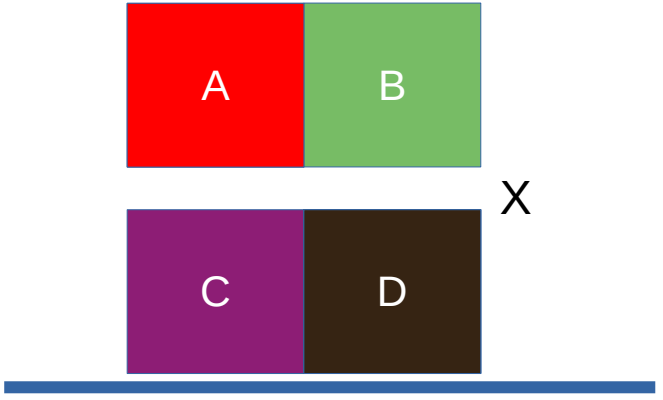
- Add blocks pairwise
- Overflow (if necessary)
- Add an extra block (if necessary)

Big Int Operations

Multiplication is harder:

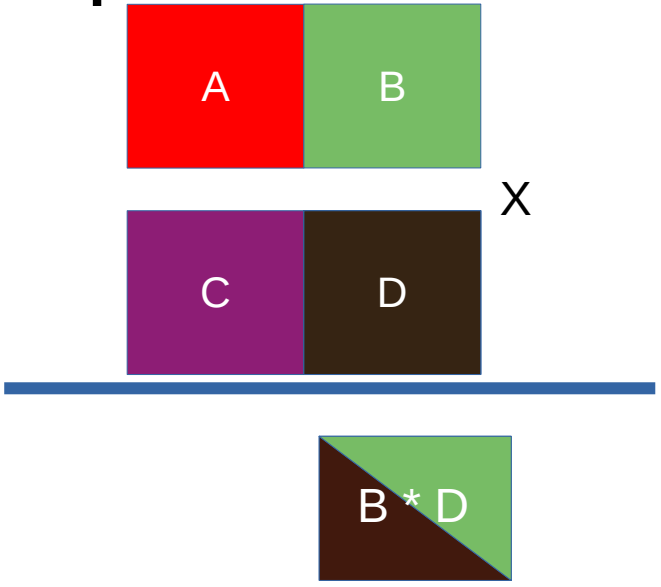
Big Int Operations

Multiplication is harder: We use long multiplication



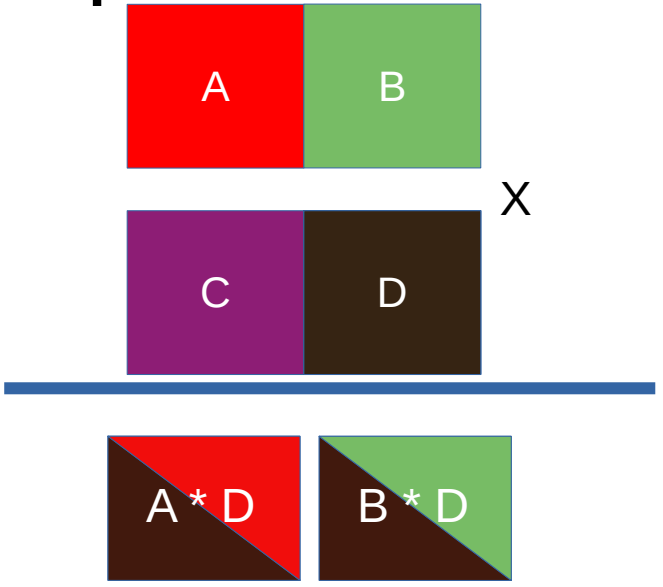
Big Int Operations

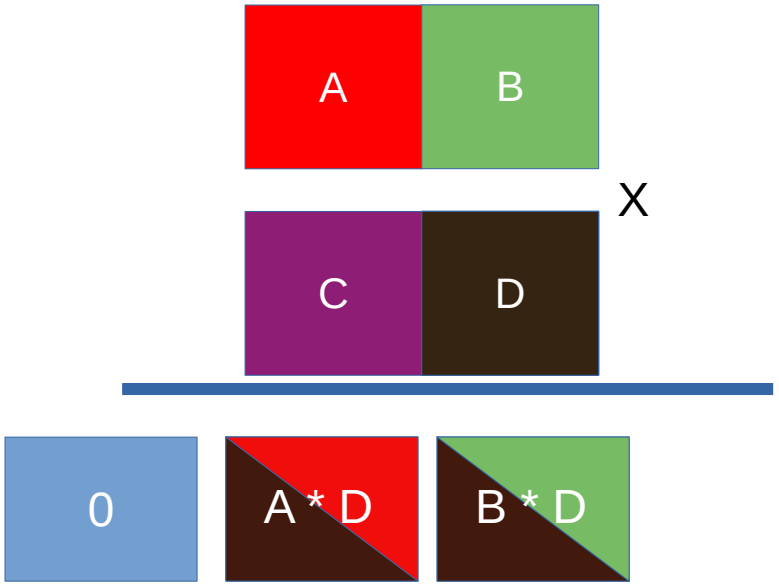
Multiplication is harder:

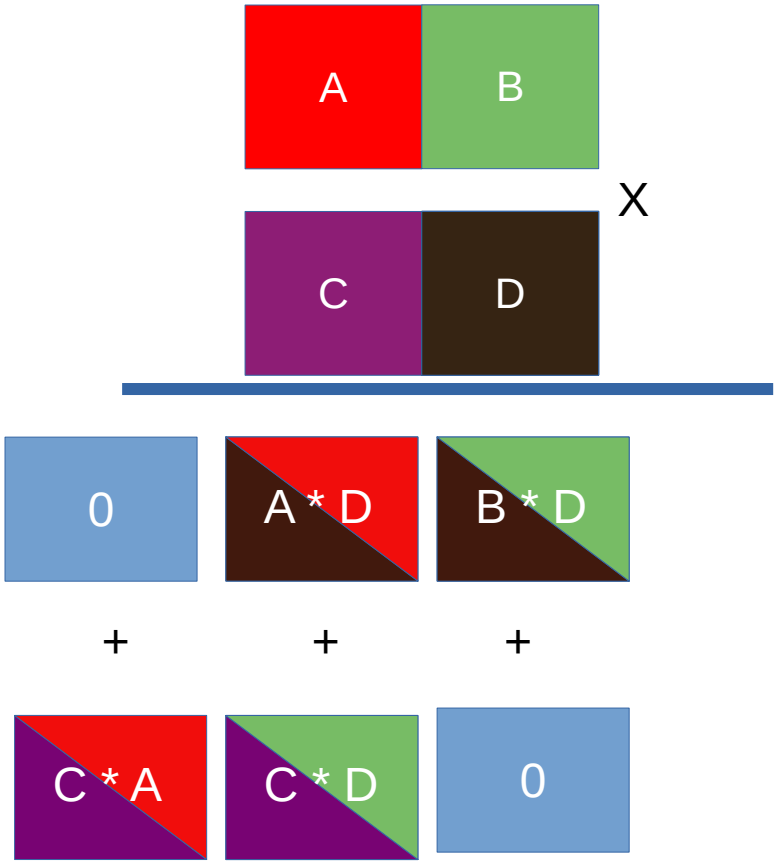


Big Int Operations

Multiplication is harder:




A
B
X
C
D
0
A * D
B * D



We need to store the result for B * D somewhere

Big Int Operations

Multiplication is harder:

2 Blocks multiplied need to fit into an already existing type

2 blocks of xbits multiplied, need a type of 2xbits, so 32bit → 64 bit

Big Int Operations

Multiplication is harder:

2 Blocks multiplied need to fit into an already existing type

2 blocks of xbits multiplied, need a type of 2xbits, so 32bit → 64 bit

Why not store the result in a 128-bit integer? (__int128)

Big Int Operations

Multiplication is harder:

2 Blocks multiplied need to fit into an already existing type

2 blocks of xbits multiplied, need a type of 2xbits, so 32bit → 64 bit

Why not store the result in a 128-bit integer? (__int128)

Not hardware optimized (unless you have an Intel APX)

Big Int Operations

Multiplication is harder:

2 Blocks multiplied need to fit into an already existing type

2 blocks of xbits multiplied, need a type of 2xbits, so 32bit → 64 bit

Why not store the result in a 128-bit integer? (__int128)

Not hardware optimized (unless you have an Intel APX)

So it is slow!!

Big Int Operations

Multiplication is harder:

2 Blocks multiplied need to fit into an already existing type

2 blocks of xbits multiplied, need a type of 2xbits, so 32bit → 64 bit

Why not store the result in a 128-bit integer? (__int128)

Not hardware optimized (unless you have an Intel APX)

So it is slow!! Doing the same thing we are down one layer deeper

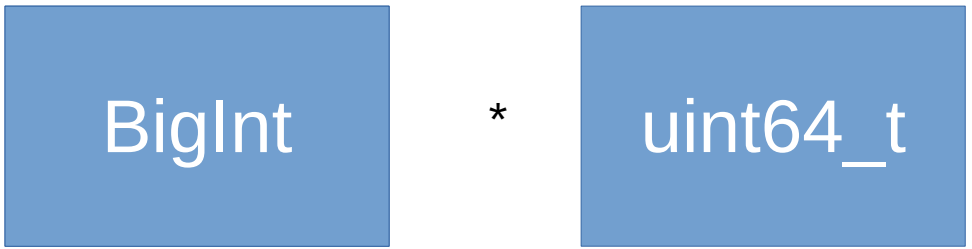
Big Float

Big Float

It’s a fixed point number!

Big Float

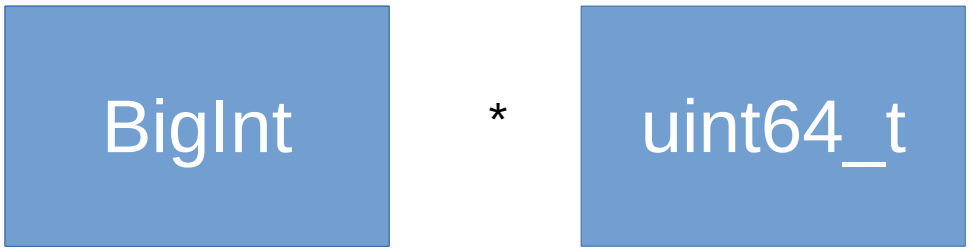
It’s a fixed point number!



value scale

Big Float

It’s a fixed point number!



value scale

Why?

Big Float

It’s a fixed point number!

Floating point operations are fast (with hardware acceleration)

Big Float

It’s a fixed point number!

Floating point operations are fast (with hardware acceleration)

- We do not have hardware acceleration

Big Float

It’s a fixed point number!

Floating point operations are fast (with hardware acceleration)

- We do not have hardware acceleration
- Float operations are 50-100x slower when using the CPU

Big Float

It’s a fixed point number!

Floating point operations are fast (with hardware acceleration)

- We do not have hardware acceleration
- Float operations are 50-100x slower when using the CPU
- We know the exact precision requirement (the user passes the amount of digits they want), **eliminates biggest benefit of floats**

Calculating PI

Calculating PI

We know that we can find Pi using this formula

$$\frac{1}{\pi} = \frac{\sqrt{10005}}{4270934400} \sum_{k=0}^{\infty} \frac{(-1)^k (6k)!(545140134k + 13591409)}{(3k)!(k!)^3 (640320)^{3k}}$$

Calculating PI

We know that we can find Pi using this formula

$$\frac{1}{\pi} = \frac{\sqrt{10005}}{4270934400} \sum_{k=0}^{\infty} \frac{(-1)^k (6k)!(545140134k + 13591409)}{(3k)!(k!)^3 (640320)^{3k}}$$

Simplified this is

$$T(k) = \frac{(-1)^k (6k)! (A + Bk)}{(3k)! (k!)^3 C^{3k}}, \qquad A = 13591409, \quad B = 545140134, \quad C = 640320$$

Calculating PI

Simplified this is

$$T(k) = \frac{(-1)^k (6k)! (A + Bk)}{(3k)! (k!)^3 C^{3k}}, \qquad A = 13591409, \quad B = 545140134, \quad C = 640320$$

Let's separate the numerator and denominator:

$$P(k) = (-1)^k (6k)! (A + Bk)$$
$$Q(k) = (3k)! (k!)^3 C^{3k}$$
$$T(k) = \frac{P(k)}{Q(k)}$$

Calculating PI

Let's separate the numerator and denominator:

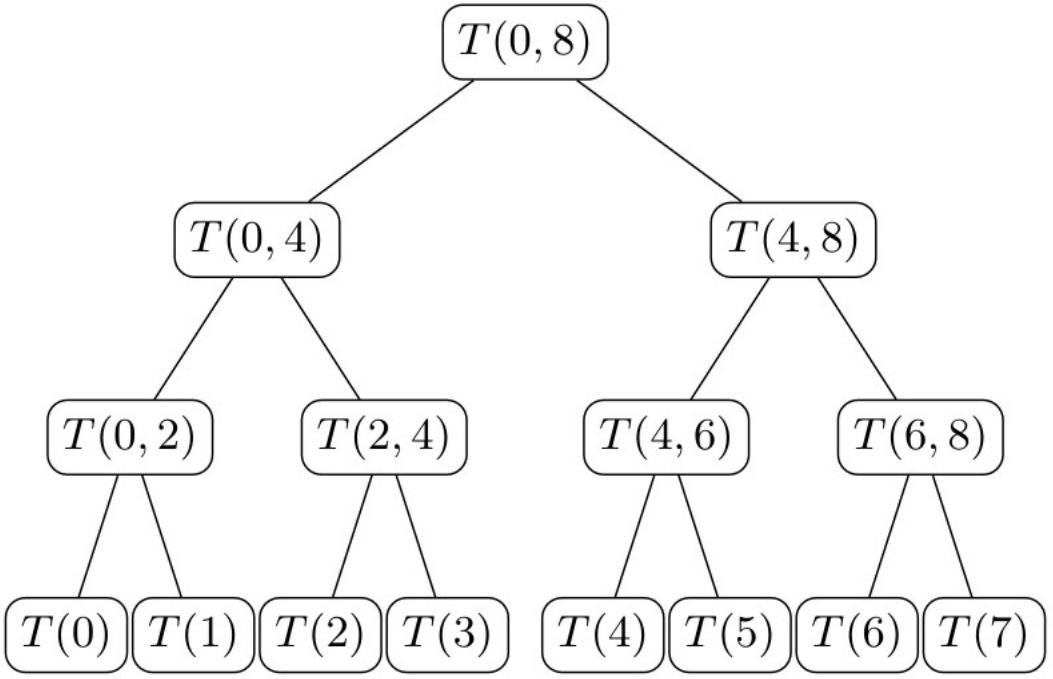
$$P(k) = (-1)^k \, (6k)! \, (A + Bk)$$
$$Q(k) = (3k)! \, (k!)^3 \, C^{3k}$$
$$T(k) = \frac{P(k)}{Q(k)}$$

Since T(k) is a sum, this means that

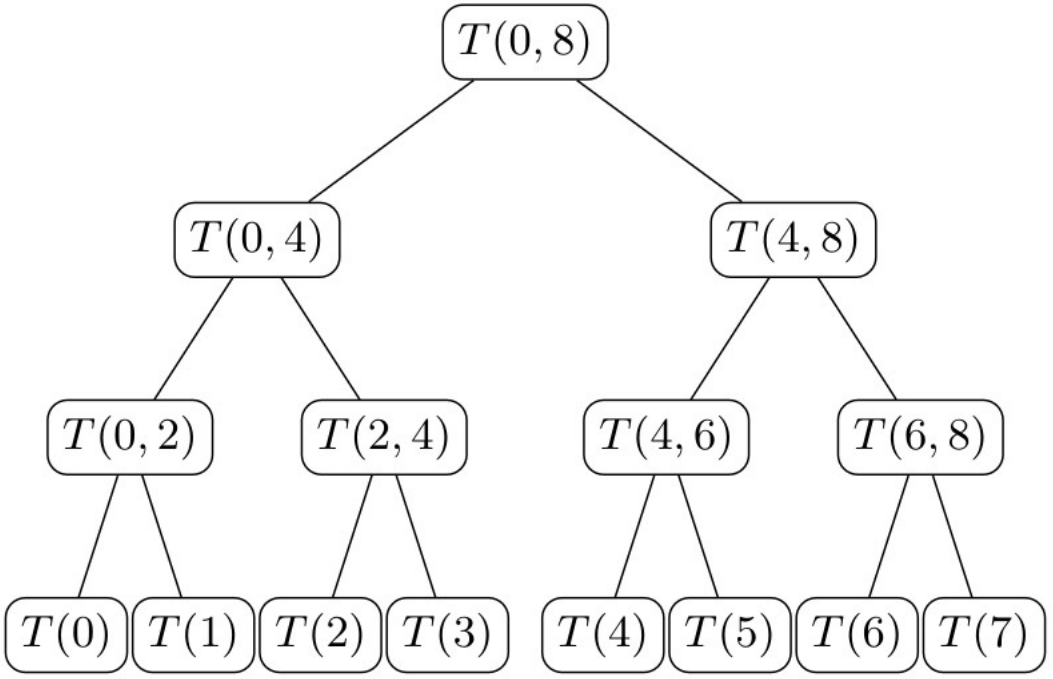
$$T(0, k) \; = \; T\left(0, \tfrac{k}{2}\right) + T\left(\tfrac{k}{2}, k\right)$$

This is recursive

Calculating PI

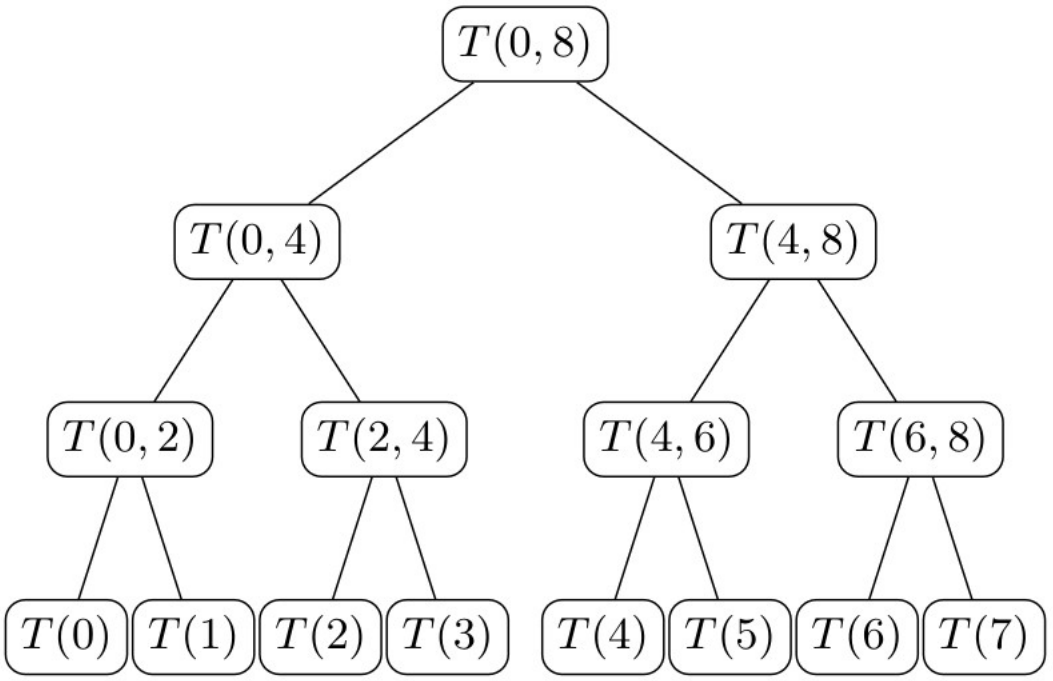


Calculating PI



But is this even faster?

Calculating PI



But is this even faster?

YES

Multiplying left to right is slow

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

Multiplying left to right is slow

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

Multiplying left to right is slow

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

144 * 4 * 7 * 3 * 6 * 5

Multiplying left to right is slow

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

144 * 4 * 7 * 3 * 6 * 5

576 * 7 * 3 * 6 * 5

Multiplying left to right is slow

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

144 * 4 * 7 * 3 * 6 * 5

576 * 7 * 3 * 6 * 5

4032 * 3 * 6 * 5

Multiplying small numbers is fast

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

Multiplying small numbers is fast

$$9 * 2 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 8 * 4 * 7 * 3 * 6 * 5$$

Multiplying small numbers is fast

$$9 * 2 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 32 * 7 * 3 * 6 * 5$$

Multiplying small numbers is fast

$$9 * 2 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 32 * 7 * 3 * 6 * 5$$

$$18 * 32 * 21 * 6 * 5$$

Multiplying small numbers is fast

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

18 * 32 * 7 * 3 * 6 * 5

18 * 32 * 21 * 6 * 5

18 * 32 * 21 * 30

Multiplying small numbers is fast

$$9 * 2 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 8 * 4 * 7 * 3 * 6 * 5$$

$$18 * 32 * 7 * 3 * 6 * 5$$

$$18 * 32 * 21 * 6 * 5$$

$$18 * 32 * 21 * 30$$

$$576 * 630$$

Multiplying small numbers is fast

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

18 * 32 * 7 * 3 * 6 * 5

18 * 32 * 21 * 6 * 5

18 * 32 * 21 * 30

576 * 630

362880

Multiplying small numbers is fast

9 * 2 * 8 * 4 * 7 * 3 * 6 * 5

18 * 8 * 4 * 7 * 3 * 6 * 5

18 * 32 * 7 * 3 * 6 * 5

18 * 32 * 21 * 6 * 5

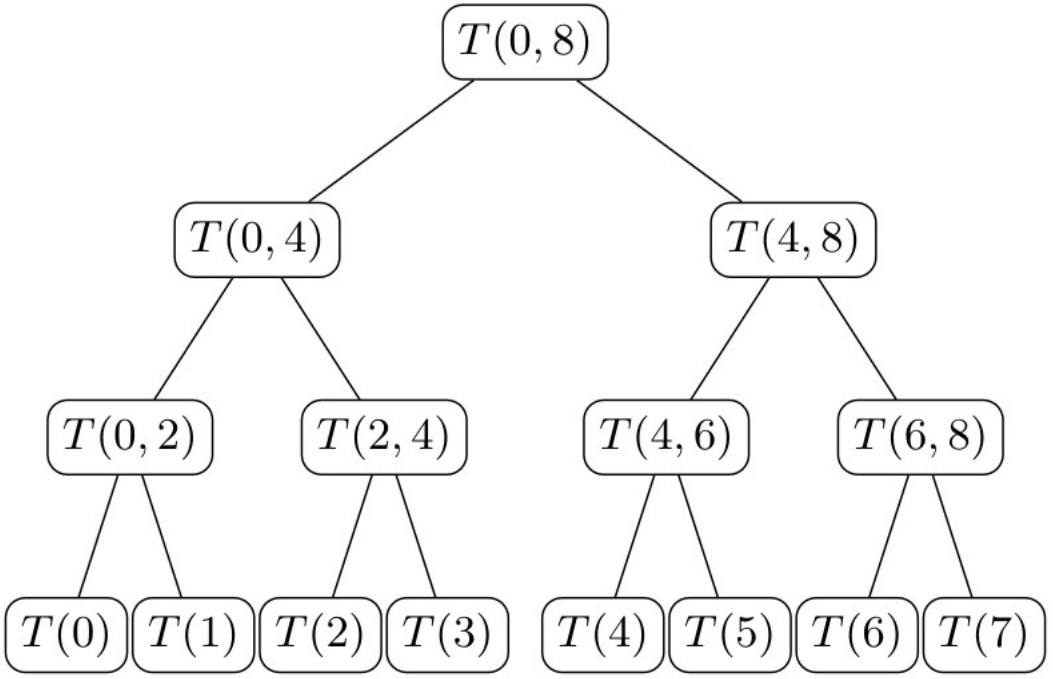
18 * 32 * 21 * 30

576 * 630

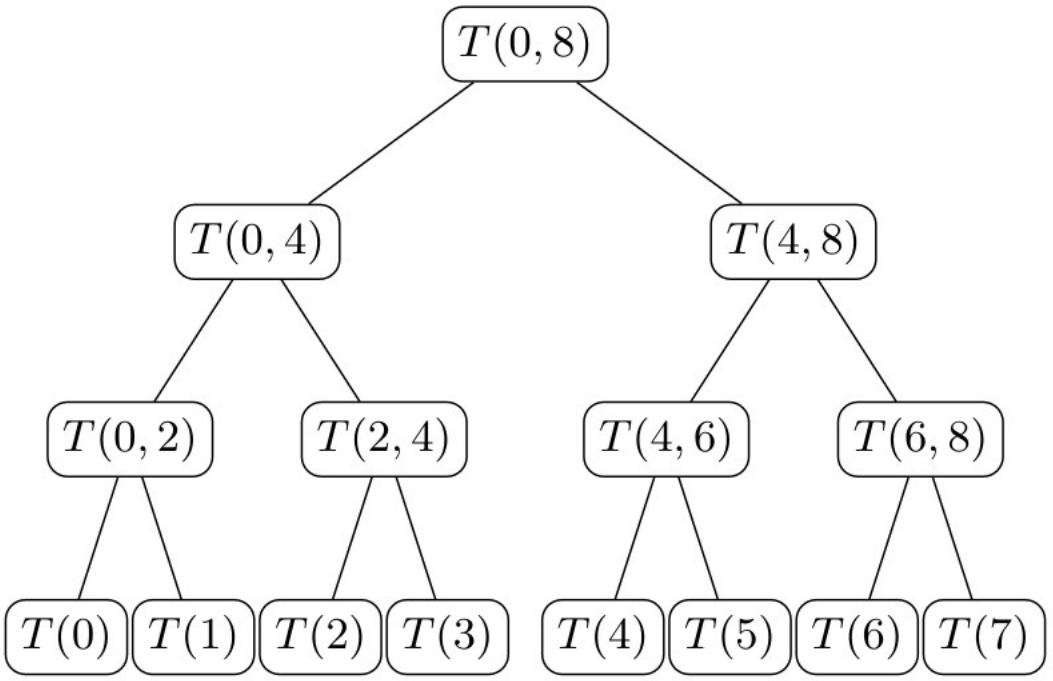
362880

Same concept applies to Binary Splitting!

Calculating PI

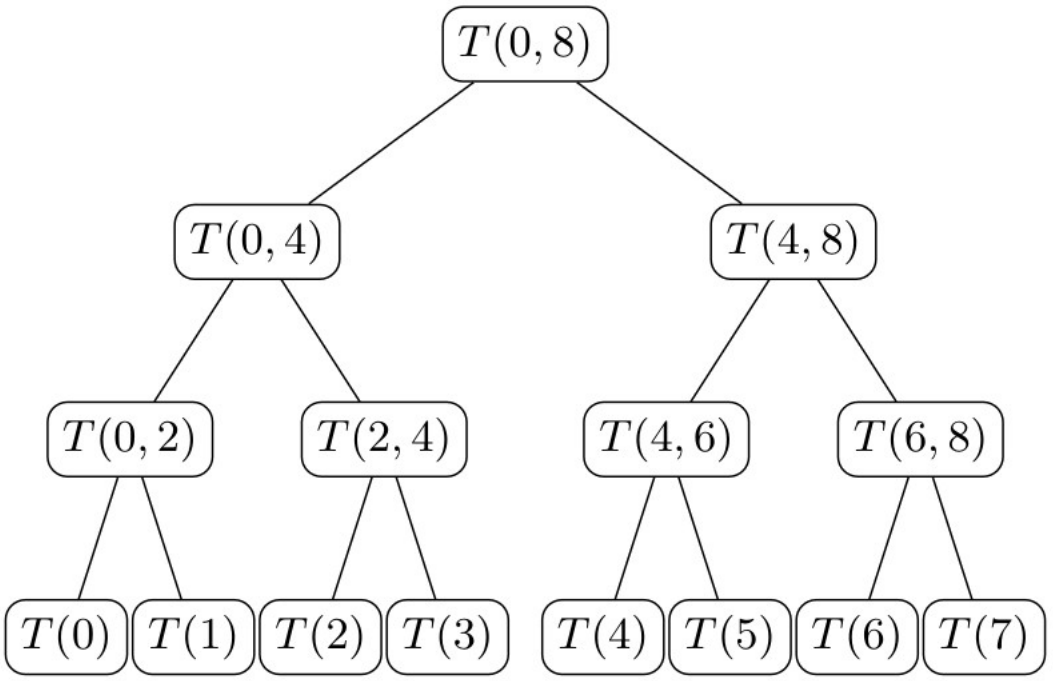


Calculating PI



Every branch is independent, and requires no info or resources from other branches

Calculating PI

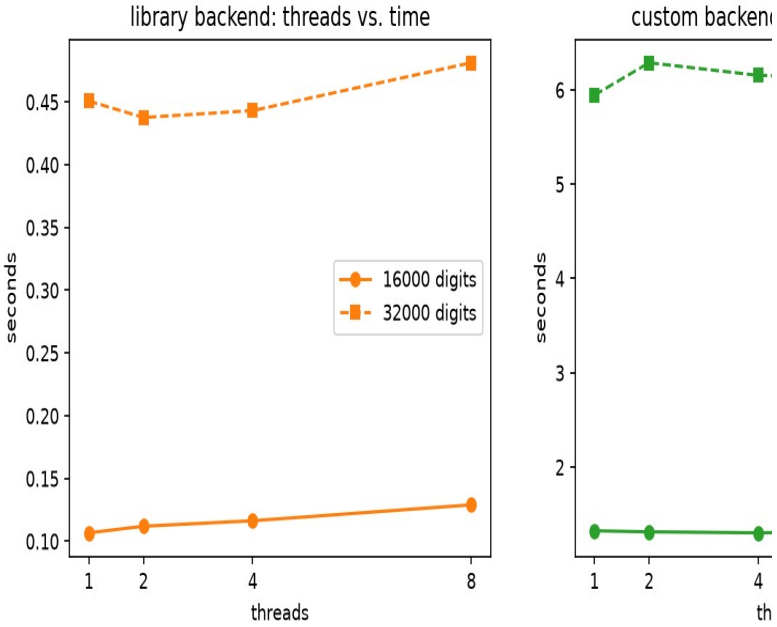


Every branch is independent, and requires no info or resources from other branches

Perfect for **multithreading**!

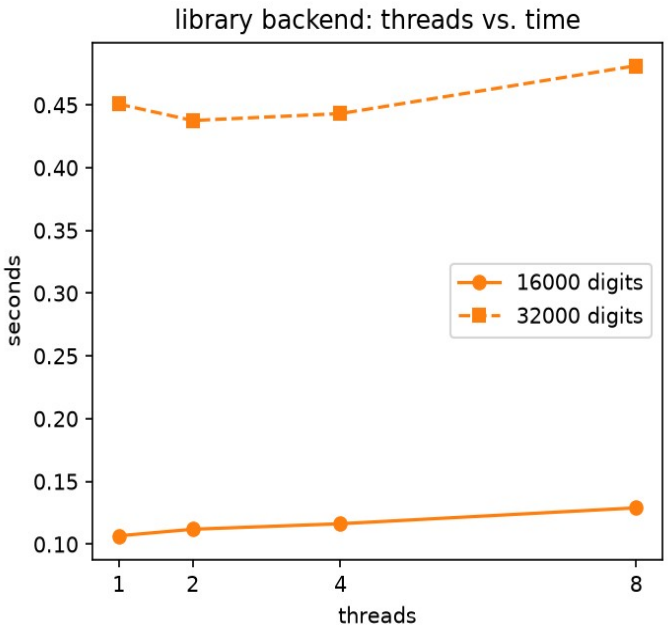
Results

Results



More threads, more runtime!

Results



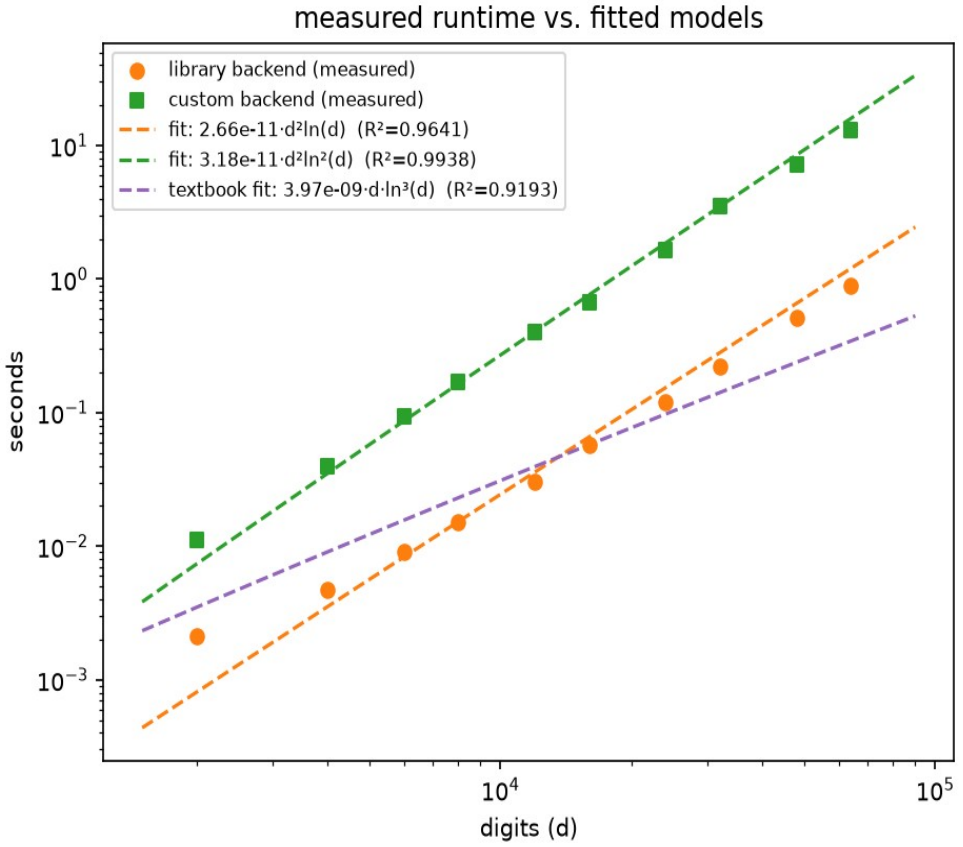
More threads, more runtime!

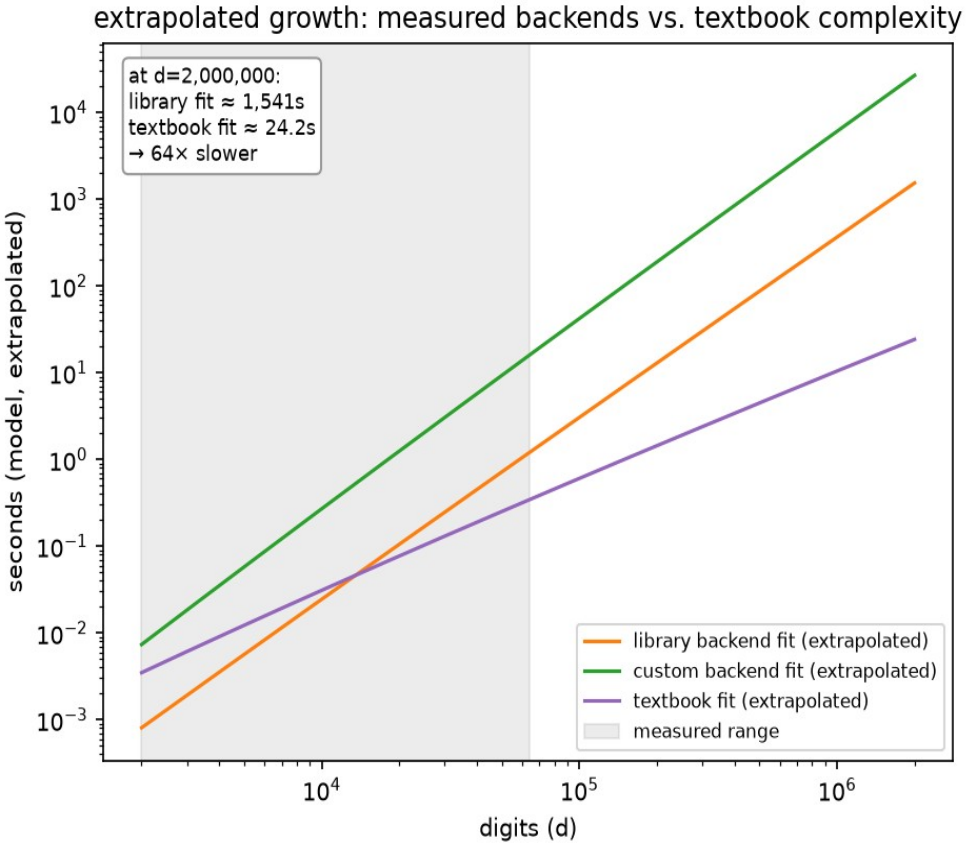
Binary splitting is **not computationally intensive**.

The majority of the runtime is **spent doing multiplication**

Multithreading multiplication is possible, but it is harder to implement, and I realized my mistake late into the project

Results


measured runtime vs. fitted models
library backend (measured)
custom backend (measured)
fit: 2.66e-11·d²ln(d) (R²=0.9641)
fit: 3.18e-11·d²ln²(d) (R²=0.9938)
textbook fit: 3.97e-09·d·ln³(d) (R²=0.9193)
seconds
digits (d)


extrapolated growth: measured backends vs. textbook complexity
at d=2,000,000:
library fit ≈ 1,541s
textbook fit ≈ 24.2s
→ 64× slower
seconds (model, extrapolated)
digits (d)
library backend fit (extrapolated)
custom backend fit (extrapolated)
textbook fit (extrapolated)
measured range

Why GMP-Chudnovsky is faster

GMP-Chudnovsky implements a different asymptotically more efficient multiplication algorithm

Why GMP-Chudnovsky is faster

GMP-Chudnovsky implements a different asymptotically more efficient multiplication algorithm

GMP-Chudnovsky actually divides by the gcd of the numbers at each binary split. This keeps numbers small.

gcd is extremely fast
division by a denominator is also fast

Low cost, high reduction in the number size!

Extras

Unit tests

Made with Catch2

31 Unit Tests

ctest --output-on-failure



Linter

Clang-tidy

LLVM Style

clang-tidy -p build --fix pi/src/chudnovsky.cpp

find pi/src -name '*.cpp' | xargs clang-tidy -p build



Documentation!

Made with DoxyGen

Check it out!

